22. ROS 2 Gazebo Simulation Platform

1. Introduction to Gazebo

Gazebo is a commonly used 3D physics simulation platform in robot development. It can simulate robots, sensors, environments, and dynamics. This tutorial uses Gazebo Harmonic (Gazebo Sim 8), which is bundled with Ubuntu 24.04 and ROS 2 Jazzy, and is integrated with ROS 2 through `ros_gz`.

Gazebo Classic is no longer maintained. The `gazebo_ros`, `spawn_entity.py`, `libgazebo_ros_init.so`, and `libgazebo_ros_factory.so` used in the Humble tutorial belong to Gazebo Classic and cannot continue to be used in the Jazzy environment of this chapter.

2. Installation and Environment Check

First load the ROS 2 Jazzy environment:

```
source /opt/ros/jazzy/setup.bash
```

Install the ROS 2 and Gazebo integration package:

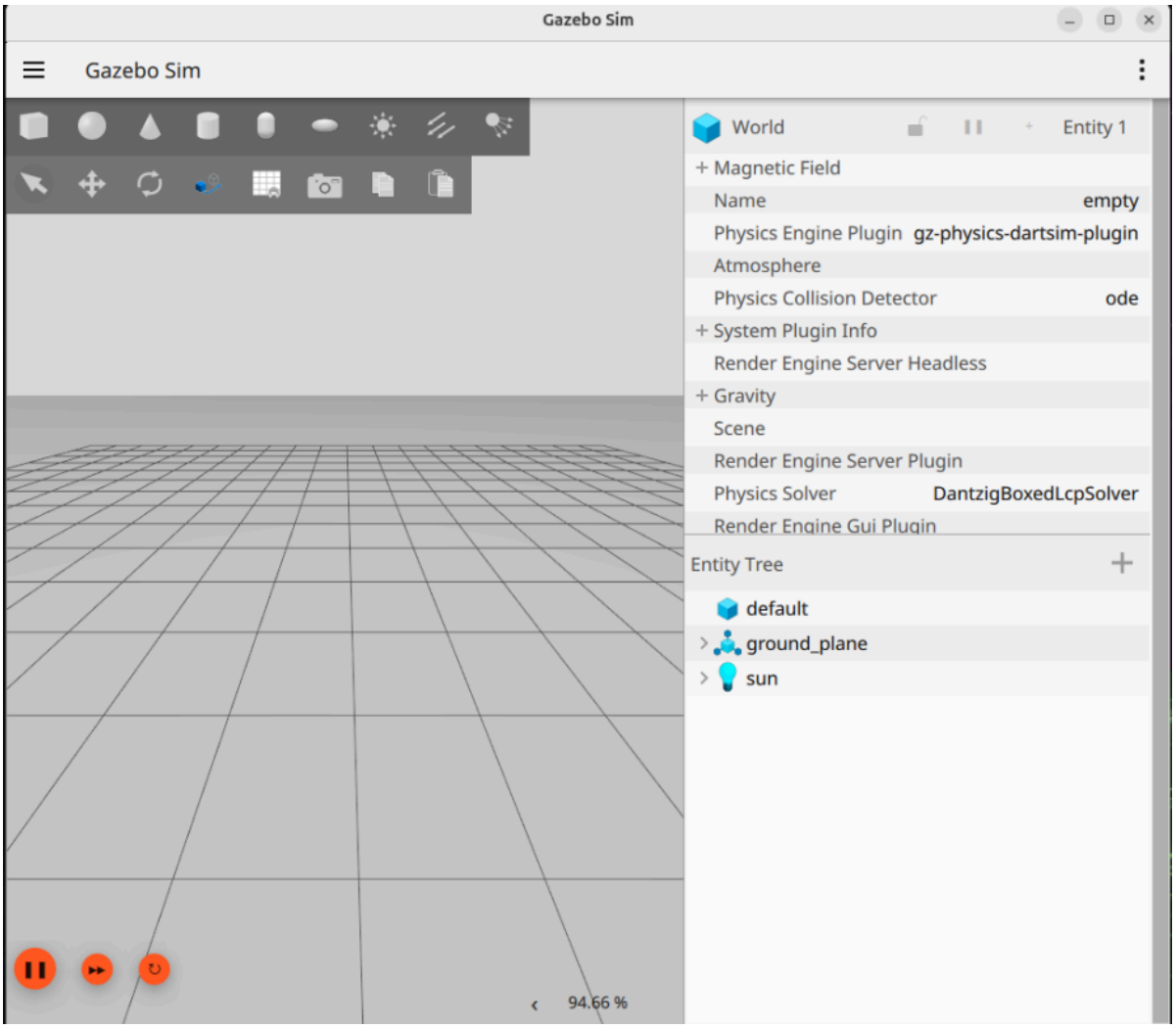
```
sudo apt update
sudo apt install ros-${ROS_DISTRO}-ros-gz
```

Check the installation result:

```
gz sim --versions
ros2 pkg prefix ros_gz_sim
```

The adapted version of this tutorial is Gazebo Sim 8 / Gazebo Harmonic. Start an empty world:

```
gz sim -r empty.sdf
```



`empty.sdf` is an empty world resource already installed with Gazebo. Do not directly execute `gz sim -s -r world.sdf` unless `world.sdf` really exists in the current directory or in the Gazebo resource path; otherwise it reports `Unable to find or download file`.

The Gazebo GUI is recommended to run in the VNC graphical desktop. When the SSH or RDP terminal does not have a correct `DISPLAY`, you can check with the server-only mode:

```
gz sim -s -r empty.sdf
```

Devices with small memory should not start multiple Gazebo instances at the same time; close other simulation worlds such as TurtleBot3 before testing this chapter.

3. Relationship Between Gazebo and ROS 2

Gazebo Harmonic internally uses Gazebo Transport for communication. When only `gz sim` is running, Gazebo nodes do not necessarily appear in `ros2 node list`, which is normal. When topics need to be passed between ROS 2 and Gazebo, `ros_gz_bridge` should be started according to the message type; this chapter uses `ros_gz_sim create` to load the model directly into Gazebo.

You can check the Gazebo entity creation service with the following command:

```
gz service -l | grep create
```

4. Creating a Package

Enter the source directory of the ROS 2 workspace and create the `myrobot` package:

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/workspace/src
cd ~/workspace/src
ros2 pkg create myrobot --build-type ament_cmake \
  --dependencies launch launch_ros ros_gz_sim
cd myrobot
mkdir -p launch urdf
```

Create `demo01_base.urdf` in the `urdf` directory. This example sets the red material directly in the `visual` of the URDF and no longer uses the `Gazebo/Red` material script of Gazebo Classic:

```
<?xml version="1.0"?>
<robot name="myrobot">
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.2 0.2 0.2"/>
      </geometry>
      <material name="red">
        <color rgba="1 0 0 1"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.2 0.2 0.2"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.0006667" ixy="0" ixz="0"
               iyy="0.0006667" iyz="0" izz="0.0006667"/>
    </inertial>
  </link>
</robot>
```

5. Writing the Launch File

Create `bringup_model.launch.py` in the `launch` directory. This launch file first calls the Gazebo launch file provided by `ros_gz_sim`, and then uses the `create` program to load the URDF model:

```
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
```

```
from launch_ros.actions import Node


def generate_launch_description():
    package_share = get_package_share_directory("myrobot")
    urdf_model_path = os.path.join(
        package_share, "urdf", "demo01_base.urdf"
    )

    gz_args = DeclareLaunchArgument(
        "gz_args",
        default_value="-r -v 3 empty.sdf",
        description="Launch arguments passed to Gazebo Sim",
    )

    start_gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory("ros_gz_sim"),
                "launch",
                "gz_sim.launch.py",
            )
        ),
        launch_arguments={"gz_args": LaunchConfiguration("gz_args")}.items(),
    )

    spawn_model = Node(
        package="ros_gz_sim",
        executable="create",
        arguments=[
            "-name", "myrobot",
            "-file", urdf_model_path,
            "-x", "0",
            "-y", "0",
            "-z", "0.2",
        ],
        output="screen",
    )

    return LaunchDescription([
        gz_args,
        start_gazebo,
        spawn_model,
    ])
```

6. Configuring Install Rules

Add the following before `ament_package()` in `CMakeLists.txt`:

```
install(
  DIRECTORY launch urdf
  DESTINATION share/${PROJECT_NAME}
)
```

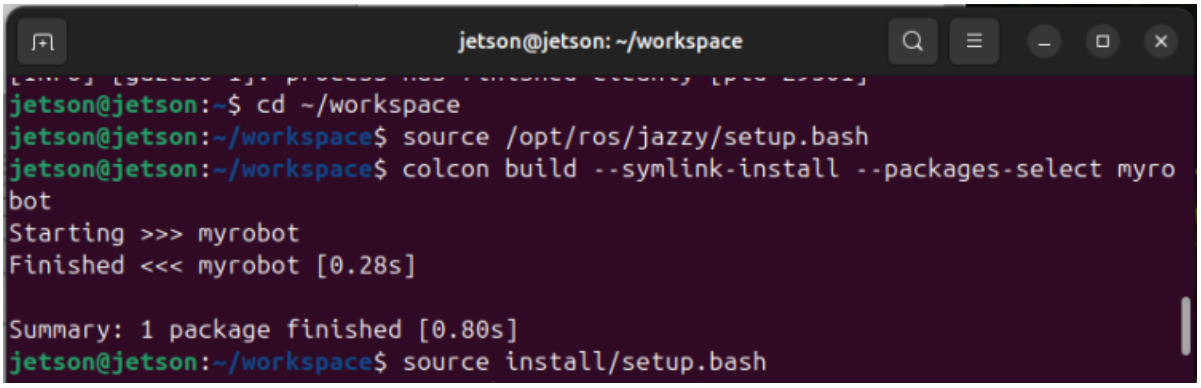
Confirm that `package.xml` contains the following runtime dependencies; they are written automatically when the package is created with the previous `ros2 pkg create --dependencies` command:

```
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>
<exec_depend>ros_gz_sim</exec_depend>
```

7. Build and Run Verification

Return to the workspace root directory and build:

```
cd ~/workspace
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select myrobot
source install/setup.bash
```



First check whether the package and the launch arguments are installed:

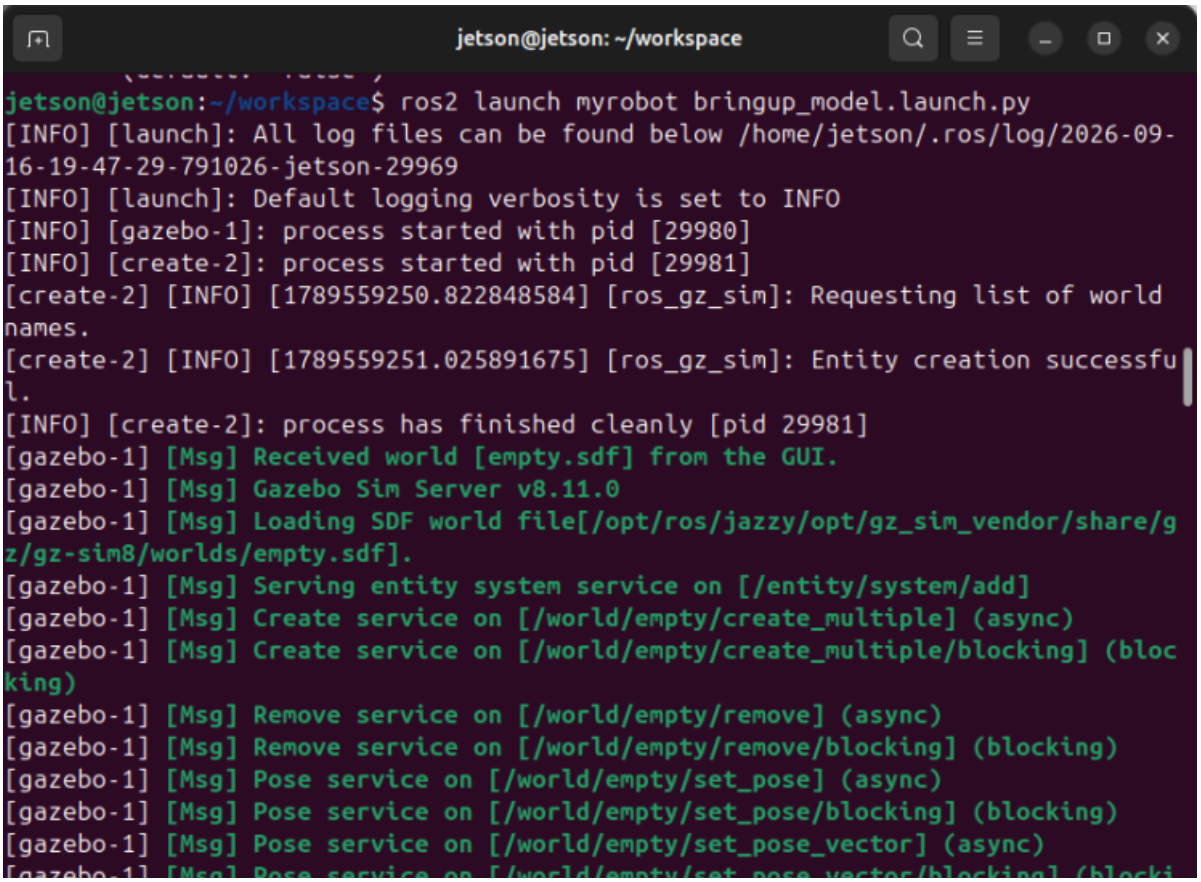
```
ros2 pkg prefix myrobot
ros2 launch myrobot bringup_model.launch.py --show-args
```

Start it in the VNC graphical desktop:

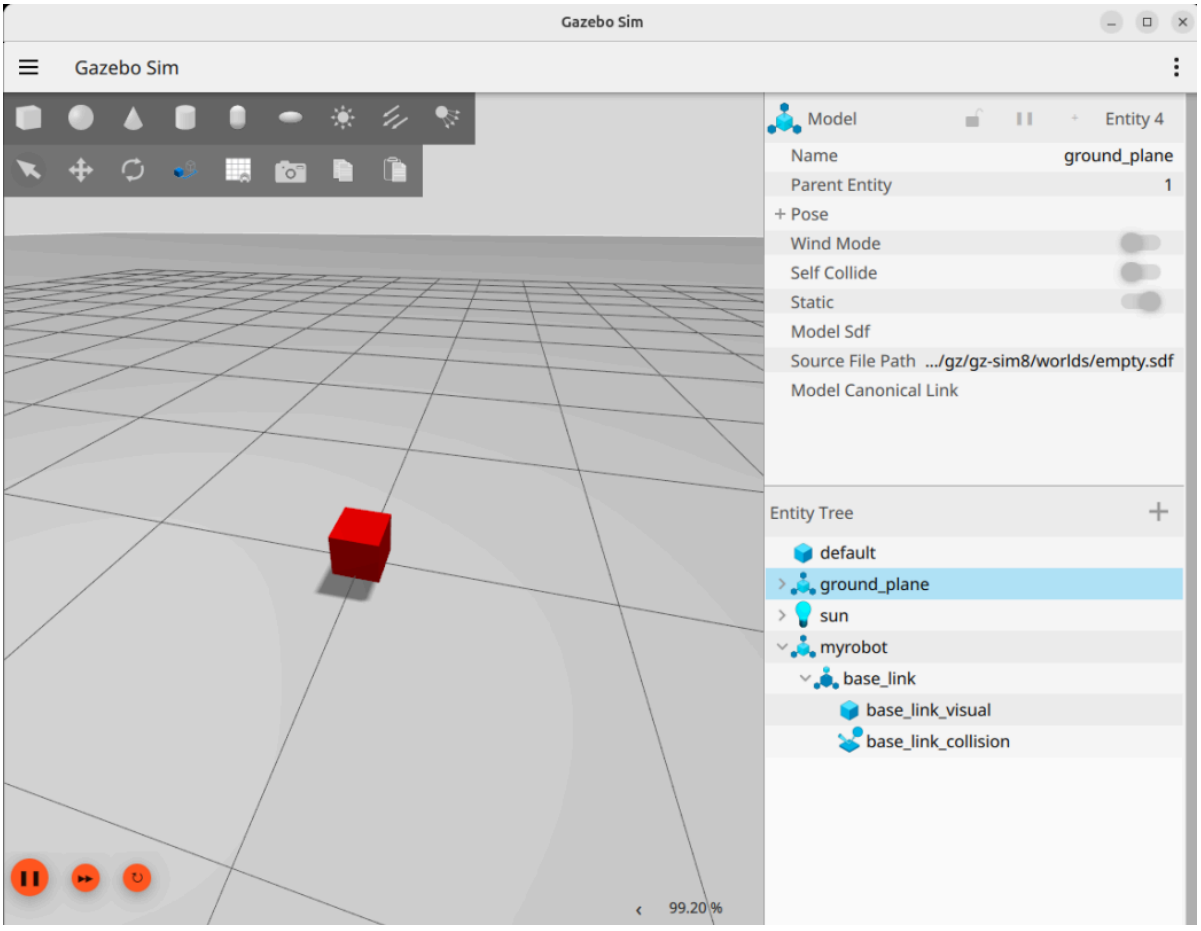
```
ros2 launch myrobot bringup_model.launch.py
```

Under normal conditions the terminal outputs:

```
[ros_gz_sim]: Entity creation successful.
```



A red cube is displayed in the Gazebo empty world. If the interface shows the TurtleBot3 world or green obstacles, it means other Gazebo processes are running. Close the other simulations first, and then start only the launch file of this chapter.



When there is no graphical desktop, you can perform a server-only verification:

```
ros2 launch myrobot bringup_model.launch.py \
  gz_args:='-s -r -v 2 empty.sdf'
```

Seeing `Entity creation successful.` means the Gazebo server has started and the model has been loaded successfully. Use `Ctrl+C` to end the program.